

Chapter 3: Processing Raw Text

This chapter covers how to work with messy text data, clean it, tokenize it, normalize it, and prepare it for NLP tasks.

Working with Files

Text files can be read using Python's `open()`. Always check encoding (UTF-8, Latin-1).

```
f = open("document.txt")
raw = f.read()
f.close()
```

String Methods

Python strings can be split, joined, replaced, and searched with methods like `split()`, `join()`, `find()`, `count()`.

```
"Monty Python".split()      # ['Monty', 'Python']
" ".join(['NLTK', 'rocks']) # 'NLTK rocks'
"text".count("t")           # 2
```

Regular Expressions

Regex helps extract and clean text using patterns. Common tokens: `\w` (word), `\d` (digit), `+`, `*`, `?` for repetition.

```
import re
re.findall(r"\w+", "NLTK, regex 101!") # ['NLTK', 'regex', '101']
re.sub(r"\d+", "NUM", "I have 2 cats") # 'I have NUM cats'
```

Tokenization

Tokenization splits sentences and words. Regex handles punctuation better than `split()`.

```
from nltk.tokenize import regexp_tokenize, sent_tokenize
regexp_tokenize("NLTK is great!", r"\w+") # ['NLTK', 'is', 'great']
sent_tokenize("Hello world. NLTK is fun.") # ['Hello world.', 'NLTK is fun.']
```

Word Normalization

Stemming crudely chops endings, while lemmatization uses vocabulary to return base forms.

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
porter = PorterStemmer()
porter.stem("running")      # 'run'
wnl = WordNetLemmatizer()
wnl.lemmatize("better", pos="a") # 'good'
```

Stopwords

Stopwords are frequent words (the, is, and) often removed to focus on meaningful words.

```
from nltk.corpus import stopwords
stopwords.words("english")[:10]
```

Collocations & N-Grams

Collocations are frequent word pairs. N-grams generalize to sequences of `n` words.

```
from nltk.util import ngrams
list(ngrams(["I", "love", "NLP"], 2))
# [('I', 'love'), ('love', 'NLP')]
```

Frequency Distributions

NLTK's `FreqDist` counts word frequencies and helps identify common terms.

```
from nltk import FreqDist
fdist = FreqDist(["dog", "cat", "dog", "fish"])
fdist.most_common(2)    # [('dog', 2), ('cat', 1)]
```

Takeaways

This chapter lays the foundation for NLP: cleaning, tokenization, normalization, stopword removal, collocations, n-grams, and frequency analysis.